나를 위한 가이드

AI 모의면접 플랫폼 — 이 코드가 **어떻게** 만들어졌고 **왜** 그렇게 만들었는지
(12살도 읽을 수 있게 풀어 쓴 설명서)

기준: 2026-11-09 · 브랜치 `ai-hiring-platform` · 커밋 전 변경 포함
작성 방식: 코드를 직접 읽고 썼습니다. 점수 계산 예시는 실제 `scoring.compute()` 를 돌려 나온 값입니다.
전체 테스트와 화면 실행은 이번에 다시 돌리지 않았습니다(맨 끝 "아직 확인 못 한 것" 참고).

0. 이 문서 읽는 법

각 부분은 같은 모양으로 쓰여 있습니다.

> **어떻게** 코드에서 실제로 어떻게 구현했는지 (파일 이름 포함)

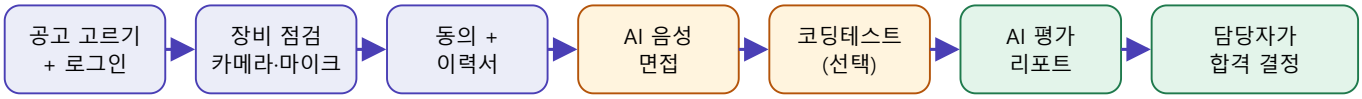
> **왜** 왜 그렇게 만들었는지, 다른 방법은 왜 안 골랐는지

> **주의** 헷갈리거나 아직 확실하지 않은 부분

1. 이 프로그램은 뭘 하나요? (한 장 요약)
2. 큰 그림 — 식당으로 이해하기 (컨테이너 8개)
3. 데이터 창고(DB)와 "한 번만 지원" 규칙
4. 로그인과 보안 — 문 열쇠와 신분증
5. 면접 진행 — 시계는 서버가 가진다
6. 평가와 점수 — AI는 의견만, 계산은 프로그램이
7. 근거 인용 검증 — "문장 번호"로 거짓말 막기
8. 작업 큐 — 번호표를 받은 일은 잊어버리지 않는다
9. 코딩테스트 — 유리방(샌드박스)
10. 질문 만들기 — 신입에게 어려운 질문을 하지 않기
11. 화면(프런트엔드) 구조
12. 내가 꼭 알아야 할 불일치·위험·아직 확인 못 한 것
13. 파일 지도와 용어 사전

1. 이 프로그램은 뭘 하나요?

회사에 지원하는 사람이 **AI 면접관**과 말로 모의면접을 하면, 프로그램이 점수와 피드백이 담긴 **리포트**를 만들어 줍니다. 기업 담당자는 그 리포트를 보고 **합격·보류·불합격을 직접** 정합니다. 모든 AI는 회사 안의 컴퓨터에서만 돌고, 외부 AI 서비스로 데이터를 보내지 않습니다.



파란색 = 지원자가 하는 일 · 주황색 = 면접 · 초록색 = 결과 처리
사람은 세 종류: 지원자 / 기업 담당자 / 운영자

그림 1. 지원자의 여정

> 왜 **AI가 합격을 정하지 않는 이유.** `CLAUDE.md` §1이 "AI는 합격·불합격을 결정하거나 추천하지 않는다"를 프로젝트 원칙으로 못 박았습니다(이유 설명은 거기에 따로 없고, 사람의 일자리가 걸린 결정이라 사람이 책임지게 한 것으로 이해하면 됩니다 — 이 문장은 제 해석입니다). AI는 "점수와 근거"만 만들고, 결정 버튼은 담당자만 누릅니다. (코드에는 점수로 계산한 "AI 예상 합격 참고값"이 담당자 화면에 있는데, 이건 모델이 정한 값이 아니라 프로그램이 `75점 이상 합격, 65~74 보류, 그 미만 불합격` 으로 나눈 것입니다 — `recruiter/router.py` . 12번 절 "불일치" 참고.)

2. 큰 그림 — 식당으로 이해하기

이 프로그램은 컨테이너(작은 가상 컴퓨터) **8개**가 함께 일합니다 (`deploy/docker-compose.yml` , `docs/container-architecture.md`). 식당에 비유하면 쉽습니다.

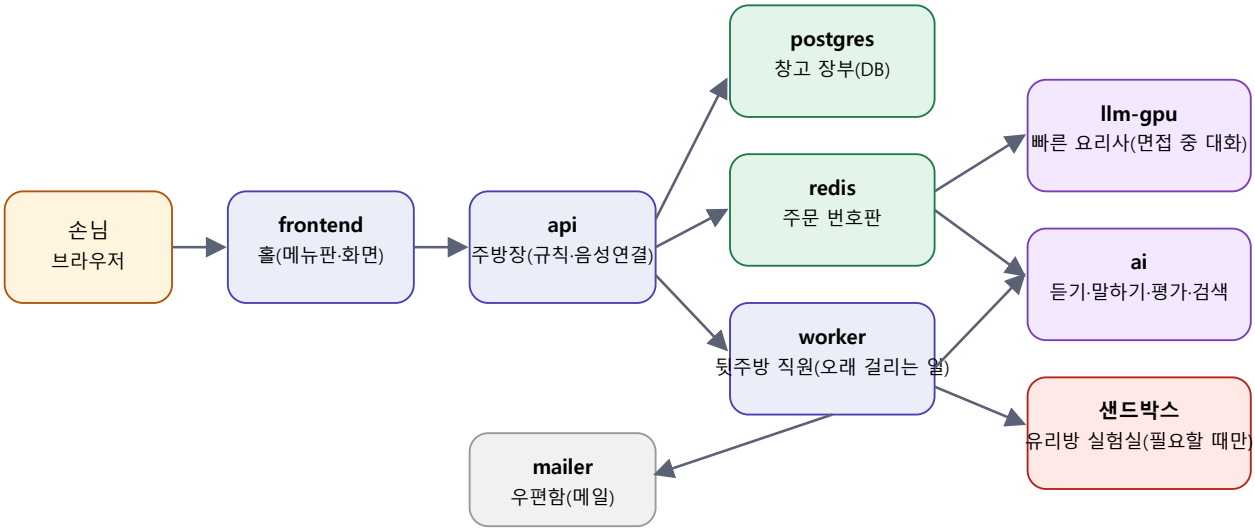


그림 2. 컨테이너 8개 + 필요할 때만 뜨는 샌드박스

컨테이너	식당 비유	하는 일 / 기술
frontend	홀과 메뉴판	웹 화면. Next.js + React 19
api	주방장	요청 처리, 로그인, 면접 진행, WebRTC(실시간 음성) 연결. FastAPI + aiortc
worker	뒷주방 직원	오래 걸리는 일(평가·질문 만들기·채점)을 번호표 순서대로 처리. Celery
postgres	창고 장부	모든 기록 저장. PostgreSQL 16 + pgvector(검색용)
redis	주문 번호판	"일 왔어요!" 알림을 빠르게 전달. Redis 7
llm-gpu	빠른 요리사	면접 중에 바로바로 답하는 언어모델(llama.cpp + Qwen3.5-4B, GPU)
ai	특별 요리사들	음성→글자(STT), 글자→음성(TTS), 평가용 LLM, 임베딩
mailer	우편함	메일 발송(개발용 Mailpit)
(샌드박스)	유리방 실험실	지원자 코드를 돌릴 때마다 잠깐 만들었다가 지움

왜 **왜 이렇게 쪼갰나?** (ADR·HANDOFF의 근거 + 제 해석) (1) 일이 오래 걸리는 평가가 면접 화면을 멈추지 않게 하려고 — 평가는 worker가 따로 합니다. (2) **AI와 코드 실행 컨테이너는 DB에 직접 접근하지 못하게** 해서, 이상한 일이 생겨도 개인정보가 새지 않게 하려고. (3) 외부 인터넷으로 나가는 길을 막아서 "이력서·답변이 외부 AI로 간다"는 일이 구조적으로 불가능하게 하려고. 이 결정은 `docs/adr/ADR-0001·0014·0015` 에 기록되어 있습니다.

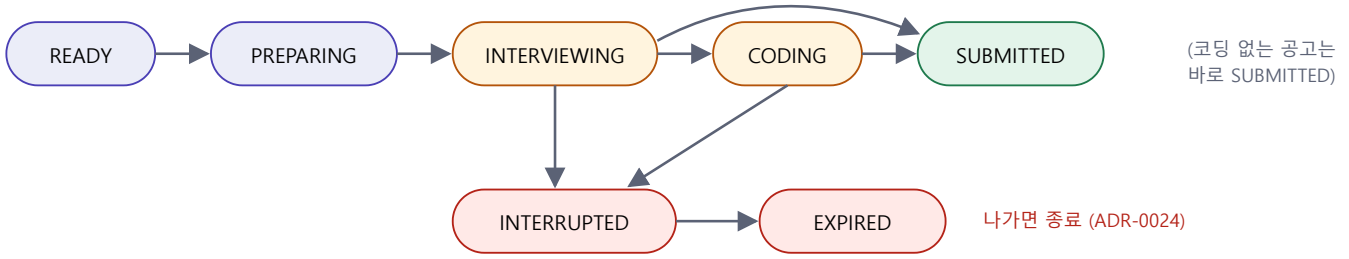
주의 6GB GPU 한 장을 여러 일이 나눠 씁니다. 그래서 "면접 중 대화용" LLM(llm-gpu)과 "평가용"을 나누고, 평가는 느긋하게(BATCH 줄) 처리합니다. 실제 속도는 이 PC에서 잰 값이며 다른 장비에서는 다를 수 있습니다.

3. 데이터 창고(DB)와 "한 번만 지원" 규칙

어떻게 같은 사람이 같은 공고에 두 번 지원하지 못하게 하는 규칙을 **데이터베이스 자체에** 걸었습니다. `migrations/versions/20261027_0001_initial.py:210` : `UNIQUE (posting_id, candidate_id)` . "한 번에 하나만 진행 중인 발언(turn)", "코딩 최종 제출은 지원 건당 정확히 1개" 같은 규칙도 같은 방식입니다 (`ux_turn_live` , `application_id uuid NOT NULL UNIQUE`).

왜 화면에서 "이미 지원했어요"라고 막는 것만으로는 부족합니다. 두 번 빠르게 누르거나 일부러 요청을 보내면 뚫립니다. **창고 문에 자물쇠를 달면** 누가 어떻게 와도 두 번째는 들어갈 수 없습니다. 이 프로젝트의 철학은 "중요한 규칙은 프런트가 아니라 서버·DB가 지킨다"입니다.

지원 건(application)이 걸어가는 상태



평가(eval_status)와 채용 결정(decision)은 이 상태와 "따로" 관리됩니다 — 평가가 실패해도 응시 기록은 안 지워집니다.

그림 3. 응시 상태. 중간에 나가면 INTERRUPTED를 거쳐 EXPIRED로 끝납니다 (`applications/lifecycle.py` 의 `end_left`)

왜 "제출 상태", "평가 상태", "합격 결정"을 **세 개의 다른 칸**으로 나눴습니다. 만약 한 칸에 다 섞으면 "평가가 실패했다 = 응시가 없던 일" 같은 사고가 납니다. 지금은 평가가 실패해도 지원자는 다시 면접을 볼 필요가 없고, 담당자가 **다시 평가** 버튼만 누르면 됩니다.

4. 로그인과 보안 — 문 열쇠와 신분증

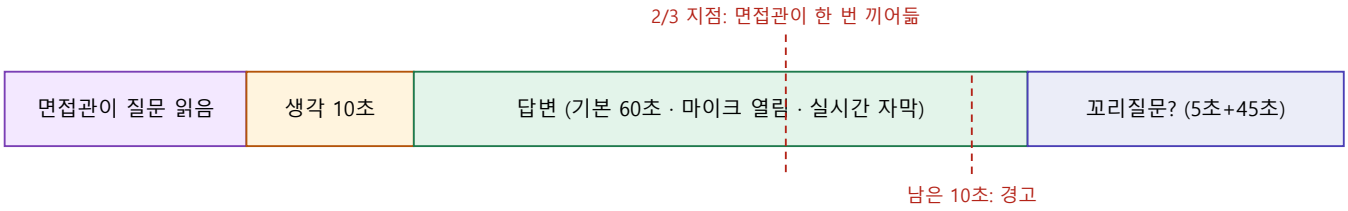
하는 일	어떻게 (파일)	왜
비밀번호 보관	Argon2id로 "해시". 원래 비밀번호는 저장하지 않음 (`auth/passwords.py`)	믹서기에 갈아 버린 과일은 원래 과일로 되돌릴 수 없는 것처럼, 장부가 유출돼도 비밀번호를 알 수 없게. 일부러 느리고 메모리를 많이 쓰게 만들어 무차별 대입을 어렵게 함
로그인 실패 제한	15분 동안 같은 이메일 5번, 같은 IP 30번 실패하면 잠금 (`auth/throttle.py`)	비밀번호를 계속 찍어 보는 공격 막기
"없는 계정"과 "틀린 비번" 구분 안 함	없는 계정도 가짜 해시와 비교(`passwords.py` 설명). 같은 오류 문구를 쓰는지는 이번에 확인하지 않음	"이 이메일은 가입되어 있다"는 사실이 새면 안 되니까
세션 쿠키	`httponly` + `samesite="strict"`, 로그인할 때마다 새 토큰 발급, 2시간 동안 활동 없으면 만료 (`auth/router.py:194`, `sessions.py`, `constants.py:57`)	자바스크립트가 쿠키를 못 읽고(XSS 방어), 다른 사이트가 대신 요청을 못 보내고, 훔친 토큰으로 이어서 쓰는 것(세션 고정)을 막음
CSRF 방어	요청에 `x-csrf-token` 헤더가 세션의 토큰과 같아야 하고, `Origin`도 검사 (`auth/deps.py`)	남이 만든 가짜 버튼으로 내 계정이 대신 행동하지 못하게
역할·소유권 검사	서버의 `require_candidate / require_recruiter / require_operator`. 화면의 `RequireRole`은 편의일 뿐 (`app/routes.tsx`)	**화면 검사는 문에 붙인 안내문, 서버 검사는 진짜 열쇠.** 주소를 직접 쳐도, 다른 사람 ID를 넣어도 서버가 막아야 함(IDOR 방지)

> 주의 "모든 API가 소유권을 제대로 검사하는지"는 이번에 하나하나 다시 확인하지 않았습니다. 코드에 검사 장치가 있다는 것까지만 확인했고, 실제 보장은 테스트(`tests/test_auth.py` 등)를 돌려 봐야 합니다.

5. 면접 진행 — 시계는 서버가 가진다

어떻게 질문 음성이 **끝난 뒤**에야 서버가 `open_answer()` 를 호출해 상태를 PRESENTING→ANSWERING으로 바꾸고, **그때 서버 시각으로** 마감 시각(`deadline_at = now + 제한시간`)을 정합니다 (`interview/turns.py:165~177`). 답변은 `finalize()` 한 곳에서 "정확히 한 번만" 확정됩니다. 버튼과 시간 만료가 동시에 와도 `on_conflict_do_nothing` 과 행 잠금(`with_for_update`)으로 한쪽만 이깁니다.

왜 브라우저의 시계는 지원자가 마음대로 바꿀 수 있습니다. 그래서 "몇 초 남았나"는 **서버만** 압니다. 화면의 카운트다운은 보여 주기용입니다. 또 질문 음성이 끝나기 전에는 마이크 입력을 받지 않아서, 질문을 듣는 도중 말한 내용이 답변에 섞이지 않습니다.



끝나는 방법은 두 가지뿐: [답변 완료] 버튼 또는 시간 만료 (조용히 있어도 저절로 끝나지 않음)

그림 4. 질문 하나의 시간 흐름 (기본값 기준)

값	어디서 정해지나	출처 파일
본질문 생각 시간	5/10/20/30초 중 공고 설정 (기본 10)	`core/constants.py` (THINK_SEC_CHOICES, DEFAULT_THINK_SEC)
본질문 답변 시간	30/60/90/120초 중 공고 설정 (기본 60)	`constants.py` (ANSWER_SEC_CHOICES, DEFAULT_ANSWER_SEC)
꼬리질문	생각 5초 + 답변 45초. 질문당 최대 1개, 공고 전체 상한 0~4(기본 2)	`constants.py` (FOLLOWUP_*, MAX_FOLLOWUPS_*, DEFAULT_FOLLOWUP_MAX)
라이브 코딩 답변	180초	`constants.py` (CODE_ANSWER_LIMIT_SEC)
끼어들기	답변 시간의 2/3가 지났는데 아직 말 중이면 1번	`constants.py` (INTERJECT_*)

주의 `CLAUDE.md` 에는 "본질문 90초·꼬리질문 60초"라고 적혀 있지만 **코드는 기본 60초/45초**이고 공고마다 바꿀 수 있습니다. 문서와 코드가 다르니 어느 쪽이 기준인지 정해야 합니다(12번 참고).

중간에 나가면?

어떻게 창을 닫거나 새로고침하거나 음성 연결이 끊기거나 신호가 120초 없으면 응시가 **INTERRUPTED→EXPIRED**로 끝납니다. 이미 답한 문항은 평가하고, 코딩테스트 중이면 마지막 저장 코드를 자동 제출합니다. "이어하기·그만두기·다시 보기"는 **일부러 없앴습니다** (`ADR-0024`, 서버가 `409 ATTEMPT_ENDED` 등으로 강제).

왜 사용자 지시(2026-11-09)가 직접적인 이유입니다. ADR-0024에는 지시 내용만 있고 이유 설명은 없습니다. (제 추측: 이어하기를 허용하면 질문을 보고 준비해서 다시 들어오는 편법이 가능해집니다.) 확실한 사실은, 서버가 강제하므로 화면을 우회해도 이어갈 수 없다는 점입니다.

6. 평가와 점수 — AI는 의견만, 계산은 프로그램이

> 왜 AI는 같은 질문에도 매번 조금 다르게 말하고, 가끔 계산을 틀립니다. 그래서 **AI는 "1~5점 중 몇 점인가"만** 말하고, 평균·가중치·총점 계산은 전부 프로그램(`evaluation/scoring.py`)이 합니다. 가중치 합이 100이 아니면 `ValueError("weights must sum to 100")` 로 저장을 거부합니다.

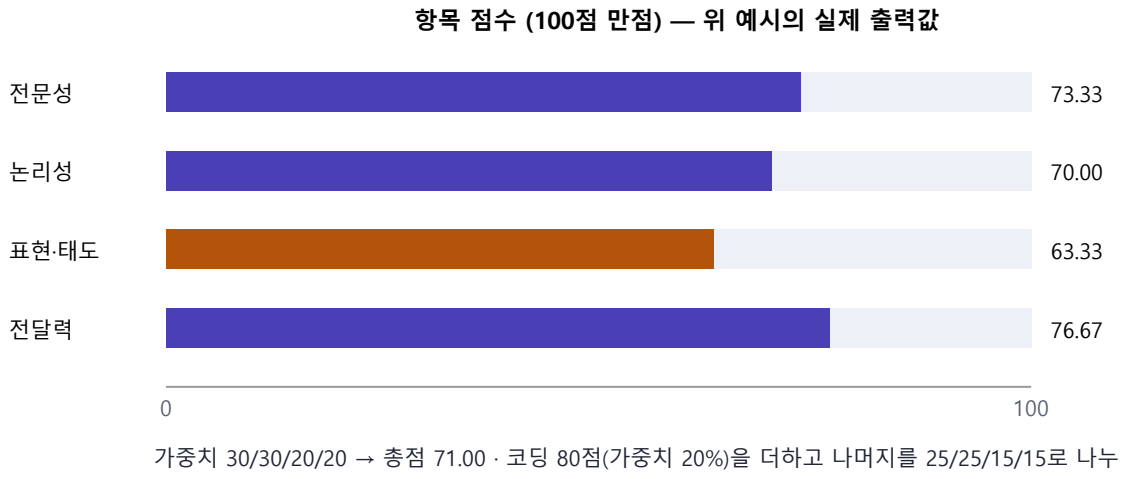
계산 순서 (코드 그대로)

1. 질문마다 4개 항목(전문성·논리성·표현 및 태도·전달력)에 AI가 1~5점. 꼬리질문이 있으면 본질문 점수와 **평균**.
2. 항목별로 모든 질문의 평균 × 20 → 100점 만점으로 바꿈.
3. 프로그램이 직접 잰 값(말 속도·발음·부정적 표현 횟수 등)이 있으면 그 항목 점수의 **30%**로 섞음 (`RULE_WEIGHT = 0.3`).
4. 항목 점수 × 가중치(%)를 모두 더해 총점. 코딩테스트가 있으면 코딩 점수 × 코딩 가중치도 더함.
5. 소수점 둘째 자리에서 **반올림(ROUND_HALF_UP)**해서 저장.

예시 — 실제 함수를 돌린 결과

세 질문의 점수(질문2에는 꼬리질문 있음)를 넣고 `scoring.compute()` 를 실제로 실행했습니다.

질문	전문성	논리성	표현·태도	전달력
Q1	4	4	3	4
Q2 본질문 / 꼬리질문	5 / 3 → **4**	4 / 3 → **3.5**	4 / 3 → **3.5**	4 / 3 → **3.5**
Q3	3	3	3	4
평균 × 20	(4+4+3)/3×20 = **73.33**	(4+3.5+3)/3×20 = **70.00**	(3+3.5+3)/3×20 = **63.33**	(4+3.5+4)/3×20 = **76.67**



가중치 30/30/20/20 → 총점 71.00 · 코딩 80점(가중치 20%)을 더하고 나머지를 25/25/15/15로 나누면 72.83

그림 5. 점수 계산 결과. 직접 계산한 값과 `compute()` 출력이 일치했습니다.

코딩 점수는 따로 계산

부분	비중	누가 정하나
정확성	60%	**테스트를 실제로 돌린 결과** (통과 수 ÷ 전체 × 100)
효율성	20%	AI가 1~5점 판단 (단, 통과가 0이면 1점, 시간·메모리 한도에 걸리면 최대 2점으로 **프로그램이 덮어씀**)
문제 해결 방식	10%	AI 1~5점
코드 품질	10%	AI 1~5점

왜 "정답인가"는 AI의 의견이 아니라 **실행해서 확인한 사실**이어야 해서 가장 큰 비중(60%)을 줬습니다 (`ADR-0017` , `scoring.py:16`).

평가 기준표(루브릭)는 어떻게 AI에게 전달되나

어떻게 `ai/rubric.py` 의 `ITEMS` 에 항목(전문성·논리성·표현 및 태도·전달력)마다 **1~5점 기준 문장(levels)**과 **점수별 예시 답변(examples)**이 있습니다. `render()` 가 이를 프롬프트로 만들어 AI에게 줍니다. 질문 유형(경험·행동·기술·설계·상황 대응)마다 "이 질문에서 보는 것" 문단이 따로 붙습니다 (`QUESTION_KINDS`). 예시 답변은 질문 유형이 바뀌어도 그대로이고, 모두 "느려진 API" 한 장면입니다.

주의 점수 판단 자체는 AI의 해석입니다. 같은 답변에도 ±1점 흔들릴 수 있고, 그래서 `ADR-0023` 에서 "독립 판정의 중앙값"을 쓰도록 했습니다. 사람 채점과 비교한 정확도 측정은 이번에 찾지 못했습니다. 논리성의 세부 항목(주장 명확성·근거 제시·논리적 흐름)은 한 줄 정의만 있고 점수별 기준·예시가 없습니다(`rubric.py:139~144`).

7. 근거 인용 검증 — "문장 번호"로 거짓말 막기

리포트에는 "이 점수는 지원자가 이렇게 말해서"라는 **근거 문장**이 붙습니다. AI는 그럴듯한 문장을 지어내거나, 한두 글자 틀리게 베껴 쓸 수 있습니다.

① 프로그램이 답변을 문장으로 자르고 번호를 붙임

[1] 응답이 3초로 느려져 로그를 확인했습니다.

[2] 주문 조회만 느려서 인덱스 누락을 의심했고,

[3] 캐시도 검토했지만 인덱스를 추가했습니다.

[4] 배포 후 0.3초로 돌아온 것을 확인했습니다.

(번호 목록만 AI에게 보여 줌)

② AI는 "번호"만 대답, 프로그램이 원문을 저장

AI 답: 논리성 4점, sent: **3**

프로그램: 3번 문장의 원문 위치(start~end)를 저장 → "캐시도 검토했지만..."

번호가 목록 밖(예: 9)이면 → 근거 아님

AI가 쓴 글자는 근거로 저장되지 않음

그림 6. 문장 번호 방식 (`evaluation/quotes.py` 의 `sentences()` / `numbered()`, ADR-0023)

어떻게 `sentences()` 가 마침표·줄바꿈 기준으로 답변을 자르고(160자 넘는 문장은 쉼표에서 더 자름, 최대 40조각), `numbered()` 가 `[1] …` 목록을 만듭니다. AI는 **번호 하나**만 돌려주고, 프로그램이 그 번호의 원문 구간(Span: 시작·끝 위치)을 저장합니다.

왜 예전에는 AI가 문장을 직접 베껴 쓰게 하고 원문과 대조했습니다(`find_span()` , 띄어쓰기·따옴표 차이는 봐주되 숫자는 정확히: "3.5배"가 "35배"로 통과하지 않음). 그런데 실제 면접에서 **인용을 비워서 내용이 있는 답변이 1점** 받는 일이 일어났습니다(ADR-0023 배경). 번호 방식은 "베껴 쓰다 틀리는" 문제 자체를 없앱니다. AI는 존재하지 않는 문장을 만들 수도, 잘못 옮길 수도 없습니다.

주의 근거를 찾지 못하면 리포트에 "근거 미연결 / 내용 부족 / 근거 추출 실패 / 답변 없음"으로 **구분해서** 표시합니다. 예전에는 근거 추출 실패만으로 최저점을 주던 것을 ADR-0023에서 바꿨습니다.

8. 작업 큐 — 번호표를 받은 일은 잊어버리지 않는다

평가·질문 만들기·코드 채점·메일 같은 일은 오래 걸려서 요청 중에 하지 않고 "작업"으로 DB에 적어 둔 뒤 worker가 가져가 처리합니다.

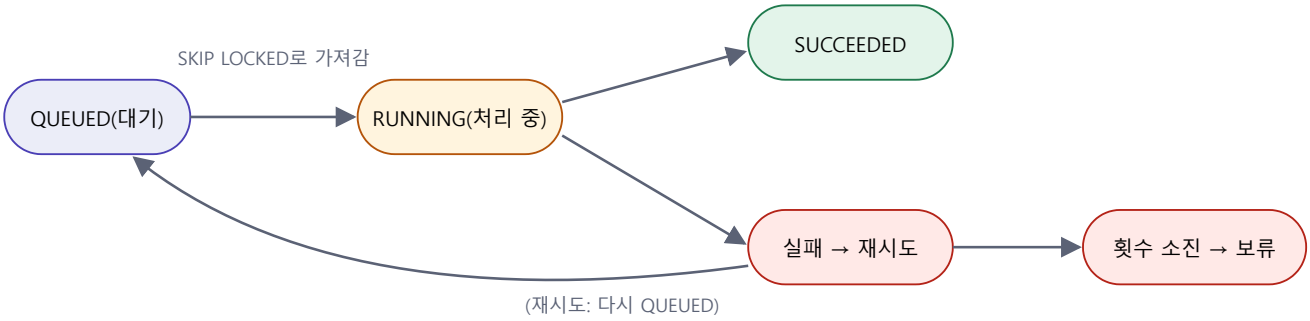


그림 7. 작업의 일생 (`jobs/worker.py` , `jobs/queue.py`)

장치	어떻게	왜
같은 트랜잭션에서 등록	`enqueue(db, …)` 는 호출한 쪽의 DB 트랜잭션 안에서 작업 행을 INSERT (`queue.py`). 예: 답변이 확정(`finalize`)되는 순간 `pre_evaluate` 작업도 같이 저장	"제출은 됐는데 평가 작업 등록은 안 됨"이라는 반쪽 상태가 생기지 않게 (둘 다 되거나 둘 다 안 되거나)
중복 방지	`dedupe_key` 가 같은 살아 있는 작업이 있으면 `ON CONFLICT DO NOTHING`	버튼을 두 번 눌러도 평가가 두 번 돌지 않게
가져가기	`FOR UPDATE SKIP LOCKED` (`worker.py:81`) — 다른 worker가 잡은 줄은 건너뜀	여러 worker가 같은 일을 동시에 집어 가지 않게
임대(lease)	가져가면 60초짜리 임대, 15초마다 연장. 연장이 끊기면 (worker 사망) 다시 가져갈 수 있음 (`worker.reap()` 이 재시작 때 회수)	worker가 죽어도 일이 영원히 "처리 중"으로 남지 않게 → **재시작 후 복구**
펜싱 토큰	가져갈 때마다 새 번호(`lease_token`)를 줌. 완료는 "내 토큰이 맞을 때만" 반영 (`fenced_complete`)	늦게 깨어난 옛 worker가 새 worker의 결과를 덮어쓰는 사고 방지
인프라 장애	모델 서버가 꺼져 있으면 worker의 "줄"이 일을 아예 가져가지 않고(probe), 횟수도 소모하지 않음 (`runtime.py` , AC-18)	서버가 꺼진 동안 일이 "실패"로 소진되면 억울하기 때문
줄(lane) 분리	LLM / LLM_EVAL / RUNNER / CPU / NONE 별로 일꾼 수를 따로 둠 (`runtime.py:24`)	오래 걸리는 평가가 질문 만들기를 막지 않게

왜 별도 큐 제품 대신 DB를 썼나? 코드 주석(`queue.py`)이 밝힌 이유는 "상태 변경과 후속 작업이 함께 커밋되게"입니다. 나머지는 제 해석입니다: 핵심 규칙(원자적 등록, 재시작 복구, 중복 방지)을 가장 간단하고 믿을 수 있게 지키려고 PostgreSQL을 "진실의 원천"으로 썼습니다. Redis는 "일 왔어요" 알림을 빨리 전하는 용도일 뿐이고, Redis가 꺼져도 DB에서 직접 가져가는 길이 있어서 일이 사라지지 않습니다 (`runtime.py _next_job` , ADR-0013).

일이 흘러가는 모습 (등록된 작업 종류)

종류	줄	우선순위(작을수록 먼저)	최대 시도
compose_questions (질문 만들기)	LLM	10	3
tts_pregen (질문 음성 미리 만들기)	CPU	15	3
extract_resume (이력서 읽기)	LLM	20	2
grade_coding (코드 채점)	RUNNER	20	3
pre_evaluate (답변별 사전 평가)	LLM_EVAL	40	2
evaluate_application (최종 평가)	LLM_EVAL	50	2

출처: `jobs/queue.py:11~22`. 그 밖에 send_email, purge_files 등이 더 있습니다.

9. 코딩테스트 — 유리방(샌드박스)

지원자가 쓴 코드는 **믿을 수 없는 코드**입니다. 컴퓨터를 망가뜨리거나 정답을 훔쳐볼 수도 있습니다. 그래서 코드를 돌릴 때마다 **새 유리방(컨테이너)**을 만들고, 끝나면 부숩니다.

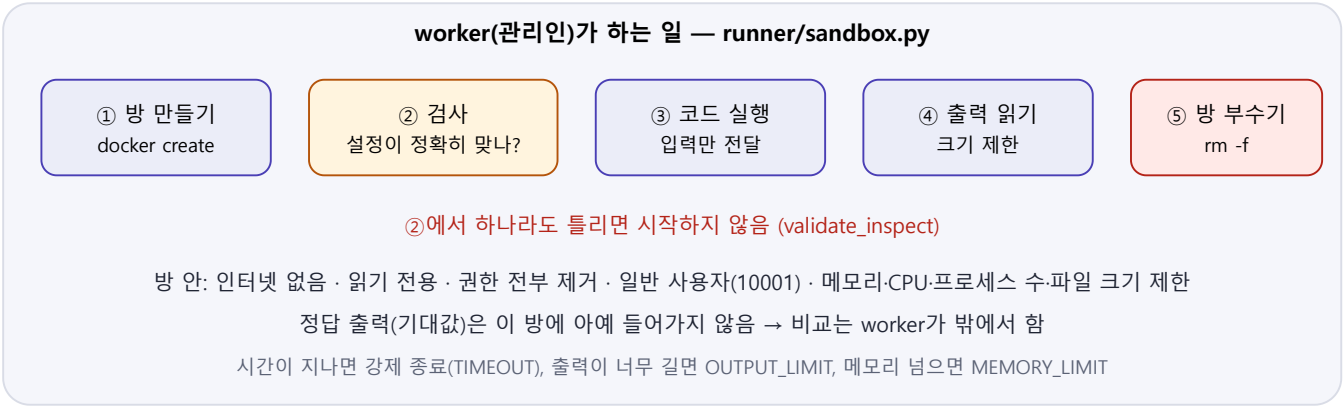


그림 8. 코드 한 건(테스트케이스 하나)을 돌리는 과정

잠금 장치 (docker create 옵션)	막는 것
`--network none`	인터넷·내부 서버로 연결 (정보 빼돌리기)
`--read-only` + `--tmpfs`	디스크에 파일 쓰기 (쓸 수 있는 곳은 임시 메모리 영역뿐)
`--cap-drop ALL`, `no-new-privileges`	관리자 권한 얻기
`--user 10001`	root로 실행
`--pids-limit`	프로세스를 무한 복제(fork 폭탄)
`--memory` = `--memory-swap`, `--cpus`, `ulimit` (nofile/fsize/core/cpu)	메모리·CPU·파일 크기 독점
`--ipc none`, `--log-driver none`	프로세스 간 몰래 통신, 로그로 정보 유출

왜 **"만든 설정이 정말 적용됐는지"를 다시 검사**합니다(`validate_inspect`). 컨테이너만 쓴다고 자동으로 안전한 게 아니기 때문입니다. 비공개 테스트케이스와 정답은 지원자 화면·API·로그·AI 프롬프트 어디에도 나오지 않고, 실행(공개 예제로만)과 최종 제출(공개+비공개 채점)을 구분합니다.

주의 (중요) `HANDOFF.md` 에 따르면 개발 PC(Windows + Docker Desktop, runtime `runc`)에서 자체 점검 SBX-01~14는 통과했지만 이는 **"개발 검증"**일 뿐입니다. 운영 Linux에서는 gVisor(`runsc`)로 다시 통과해야 합니다. 이번에 다시 돌려 확인하지는 않았습니다. 그 전에는 임의 코드 실행을 외부에 열면 안 됩니다.

10. 질문 만들기 — 신입에게 어려운 질문을 하지 않기

어떻게 `questions/level.py`가 이력서의 **일한 기간**을 더해(인턴·알바·교육은 제외) 수준을 정합니다. 1년 미만 NEW, 1~5년 JUNIOR, 5년 이상 SENIOR (`career_level`, 65행). 공고가 "신입"만 원하면 SENIOR를 JUNIOR로 낮춥니다. 그다음 `difficulty_plan()`이 "기초 : 실무 적용 : 심화" 비율로 질문 난이도를 나눕니다 (`MIX`, 24행).

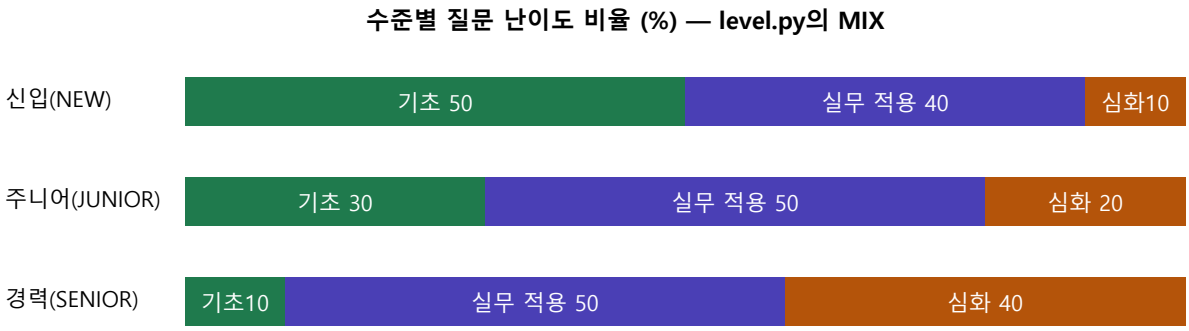


그림 9. 지원자 수준에 따른 질문 난이도 (코드의 `MIX` 값 0.5/0.4/0.1 등을 %로 표시)

왜 실제 면접 기록에서 신입에게 Kafka·Redis·Prometheus를 함께 엮은 설계 질문이 나갔고, 지원 동기 칸에서 시스템 아키텍처를 묻는 일이 있었습니다(`ADR-0025` 배경). 그래서 (1) 수준을 코드가 계산하고, (2) 질문이 나오면 코드가 다시 검사합니다 — 기술이 3개 이상 섞였거나, 신입에게 심화 복잡도면 거절 (`problems()`, `fits_level()`). AI에게 "신입에게 쉽게 물어봐"라고만 부탁하지 않고 **프로그램이 직접 확인**하는 것이 핵심입니다.

질문을 언제 만드나? (upfront vs live)

어떻게 `core/config.py:55`의 `question_mode` 기본값은 **"upfront"**(면접 시작 전에 10개를 미리 확정)입니다. "live"(질문을 면접 중에 즉석에서 만들기)는 설정으로 남겨 두었습니다.

왜 `ADR-0025`에 따르면 6GB GPU에서 질문 하나를 만들 때 프롬프트 처리만 약 6초, 검증에 실패해 재시도하면 두 배가 걸려서 질문 사이 대기가 **10~40초**가 되었습니다. 미리 만들어 두면 면접 중 끊김이 없고, PRD 원래 규칙(면접 전에 본질문 확정, 응시 시점의 불변 스냅샷)과도 맞습니다. 이 수치는 문서에 적힌 측정값이며 이번에 다시 재지는 않았습니다.

11. 화면(프런트엔드) 구조

주제	어떻게	왜
화면 라우팅	`app/routes.tsx`에 한 표로 정리. 누구나 / 로그인 필요(`RequireAuth`) / 역할 필요(`RequireRole`) 3겹	어느 주소에 누가 들어오는지 한눈에 보려고
기업 화면 분리 로딩	기업 담당자 화면은 `lazy(() => import(...))`로 따로 내려받음	지원자 브라우저는 기업 화면 코드를 아예 받지 않음 (필요 없는 코드를 안 보냄)
면접 진행 상태 기계	`candidate/interview/machine.ts` — 순수 함수(reducer)로 "연결중→생각→답변→정리…" 상태를 계산	화면 없이도 테스트할 수 있고, 상태가 꼬이지 않음 (`machine.test.ts`)
장비 점검	`apply/Check.tsx` — 카메라·마이크·스피커가 모두 "정상"이어야 버튼 활성화, 실패 이유를 사람 말로 번역(`deviceProblem`)	"왜 안 되는지" 알려 줘야 지원자가 스스로 고칠 수 있어서
카메라	본인 화면 표시용으로만 사용. 서버로 보내거나 저장하는 코드 없음	개인정보 최소화 원칙(영상 저장 금지)
면접 완료 → 리포트	`Done.tsx`가 5초마다 평가 상태를 물어봄(polling), DONE이 되면 멈춤	평가는 worker가 따로 하므로 "다 됐나?"를 주기적으로 확인
접근성	`role="alert"`, `aria-live`, 라벨 등 사용	스크린리더 사용자도 쓸 수 있게

12. 내가 꼭 알아야 할 불일치·위험·아직 확인 못 한 것

아래는 이번에 코드를 읽으며 **직접 확인한 불일치**와 **확인하지 못한 것**입니다. 문서를 믿기 전에 한 번 보세요.

(가) 문서와 코드가 다른 곳

#	내용	어디
1	평가 버전: ADR-0023은 `evaluate_answer@2.0.0 / rubric-v4`, **코드는 `2.1.0 / rubric-v5`**	`ai/tasks/evaluate.py:107`, `ai/rubric.py:25`
2	`postings/schemas.py:13`에 `RUBRIC_VERSION = "rubric-v3"`가 따로 남아 있음. 의도된 것인지(공고 설정용?) 낡은 값인지 확인 못 함	`postings/schemas.py:13`
3	`CLAUDE.md`: 본질문 90초 / 꼬리질문 60초. **코드: 기본 60초 / 45초** (공고별로 30~120초 선택)	`core/constants.py`
4	`CLAUDE.md` §0은 "단일 관리자, 멀티테넌시 제외". **코드는 기업(company) 계정·운영자 인증 절차가 구현**되어 있음	`auth/company_signup.py`, `screens/operator/`
5	`CLAUDE.md` §1: "AI는 합격·불합격을 추천하지 않는다". 코드에는 담당자 화면에 "AI 예상 합격(합격 추천/불합격 추천)" 표시가 있음. 모델이 아닌 점수 구간(75/65)으로 계산하지만 문구는 "추천". 화면정의서 결정이라는 메모(memory)가 있으나 **CLAUDE.md와 충돌**하므로 최종 판단이 필요	`recruiter/router.py:94`, `Applications.tsx:8`
6	지원 현황 화면 주석은 "점수는 절대 보이지 않는다(SEC-04)"인데, 지원자 리포트 API는 총점·등급·항목 점수를 돌려줌	`MyApplications.tsx:21` vs `applications/report.py` (ReportOut의 total, grade, criteria)
7	문서 일부는 옛 상태: `HANDOFF.md`는 2026-10-28 기준(예: 재연결 1·2·4초 후 텍스트 모드는 ADR-0024에서 삭제됨)	`HANDOFF.md`

(나) 이번에 하지 않은 것 (미검증)

- **테스트 실행 안 함**: 백엔드 pytest, 프런트 vitest, runner 테스트 모두 이번에 돌리지 않았습니다. 메모리 상의 "568 통과"는 이전 기록이며 제가 확인한 값이 아닙니다.
- **화면 실행·캡처 안 함**: 가이드의 화면 설명은 소스 코드의 문구를 읽고 쓴 것입니다.
- **GPU 성능·다중 사용자 부하·외부망 WebRTC(TURN)**: 실측하지 않았습니다.
- **샌드박스 운영 검증(gVisor)**: 개발 PC 검증만 있습니다.
- **소유권(IDOR) 전수 점검**: 모든 API를 하나씩 보지 않았습니다.
- **ADR-0024·0025·0023 서술을 코드와 줄 단위로 대조하지 않음**: 이 가이드에서 코드로 직접 확인한 것은 점수 계산, 인용 문장 번호화, 작업 큐, 샌드박스 옵션, 로그인·세션, 시간 상수, 질문 수준 비율, 답변 확정 로직입니다.

위험 작업 폴더에 **커밋 전 변경이 74개 파일**(수정 60·삭제 1·신규 13)입니다. 새 ADR 3개, 마이그레이션 0033, 테스트 6개가 포함됩니다. 커밋하기 전에는 이 변경이 어디에도 안전하게 저장되어 있지 않습니다. 커밋은 "커밋"이라고 말씀하실 때만 하도록 설정되어 있습니다.

13. 파일 지도와 용어 사전

어디를 보면 되나

알고 싶은 것	파일
점수 계산	`backend/app/evaluation/scoring.py`
인용(근거) 검증·문장 번호	`backend/app/evaluation/quotes.py`
평가 흐름 전체	`backend/app/evaluation/pipeline.py`, `backend/app/ai/tasks/evaluate.py`
평가 기준표·프롬프트	`backend/app/ai/rubric.py`
면접 진행(시간·답변 확정)	`backend/app/interview/turns.py`, `interview/router.py`
실시간 음성 (WebRTC)	`backend/app/voice/session.py`
작업 큐	`backend/app/jobs/queue.py` (등록), `worker.py` (가져가기), `runtime.py` (줄·정기작업)
코딩 채점·샌드박스	`backend/app/coding/`, `runner/runner/sandbox.py`
질문 만들기·수준	`backend/app/questions/compose.py`, `level.py`
로그인·세션	`backend/app/auth/`
화면	`frontend/src/screens/{candidate,recruiter,operator,common}/`, 라우트 `frontend/src/app/routes.tsx`
결정 기록(왜 이렇게?)	`docs/adr/ADR-0001 ~ 0025`
요구사항 ↔ 구현 ↔ 테스트 표	`docs/requirements-traceability.md`

용어 사전

말	쉬운 설명
API	화면이 서버에게 "이거 해 줘" 하고 보내는 주문서의 약속된 형식
DB 제약(UNIQUE)	창고 장부에 건 자물쇠. 규칙을 어기는 기록은 아예 적히지 않음
해시(Argon2id)	되돌릴 수 없게 갈아 버린 비밀번호
CSRF	남이 만든 가짜 버튼으로 내 이름으로 행동하게 만드는 공격
IDOR	주소의 번호만 바꿔서 남의 자료를 보는 공격
WebRTC	브라우저와 서버가 실시간으로 소리를 주고받는 기술(전화 같은 것)
STT / TTS	음성→글자 / 글자→음성
LLM	글을 이해하고 쓰는 AI (여기서는 사내 서버에서 도는 Qwen3.5-4B)
임베딩	글을 숫자 좌표로 바꿔서 "비슷한 글" 찾기에 쓰는 기술
루브릭	점수 기준표 ("5점은 이런 답, 3점은 이런 답")
트랜잭션	"모두 성공하거나 모두 취소"되는 묶음 작업
멱등(idempotent)	같은 일을 두 번 해도 결과가 한 번 한 것과 같음
lease(임대) / fencing	"이 일은 내가 60초 맡을게요" / 늦게 온 옛 담당자가 결과를 덮어쓰지 못하게 하는 번호표
샌드박스	모래상자·유리방. 위험한 코드를 가두고 돌리는 격리 환경
ADR	"왜 이렇게 정했나"를 적어 둔 결정 기록 문서
upfront / live	질문을 면접 전에 미리 / 면접 중에 즉석에서 만들기
연습 모드	지원자가 "결과 공유"에 동의하지 않은 응시. 기업에는 보이지 않음

이 문서는 `docs/guides/04-개발자-가이드.html`에서 만들어진 PDF입니다. 지원자·기업 담당자·운영자 가이드는 같은 폴더의 `01~03` 문서를 보세요.